

Projeto Integrador

Documentação Técnica – Simulador de Termografia Mamária

Aluno[6]: Diego Ferreira

Turma[20]: Água Fria.

Github do projeto: [clique aqui](#)

1. INTRODUÇÃO

Este projeto foi desenvolvido com o objetivo de criar um **simulador de termografia mamária** utilizando o framework **Django**.

Como não é viável utilizar dados reais de pacientes por questões éticas e legais, o sistema foi projetado para **gerar dados fictícios**, imagens simuladas e classificações automáticas aleatórias, permitindo a realização de testes e estudos sem risco à privacidade.

O simulador representa um ambiente controlado onde é possível:

- Criar pacientes fictícios
- Gerar imagens de termografia simuladas
- Simular respostas de um classificador inteligente (inspirado no WEKA)
- Consumir essas informações por meio de endpoints HTTP

2. OBJETIVOS

Objetivo Geral

Desenvolver um **simulador de termografia mamária** capaz de gerar dados e resultados fictícios de forma automatizada, permitindo testes, validações e aprendizado sobre sistemas médicos computacionais.

Objetivos Específicos

- Criar um projeto Django organizado e funcional
- Desenvolver um sistema de geração de **pacientes fictícios** utilizando a biblioteca Faker
- Simular a criação de **imagens de termografia mamária** usando técnicas de processamento de imagem
- Implementar um **gerador de respostas aleatórias**, simulando a classificação de um sistema inteligente
- Criar **endpoints (URLs)** para acessar as funcionalidades do simulador
- Desenvolver um **comando de gerenciamento** para popular o banco de dados automaticamente

- Documentar todo o processo de forma clara e didática
-

3. PLANEJAMENTO

Antes do desenvolvimento, foi realizado um planejamento básico dividido em etapas, para garantir organização e entendimento do projeto.

O planejamento seguiu a seguinte ordem:

1. Criação do projeto e dos aplicativos Django
2. Definição da estrutura de pastas
3. Implementação da geração de pacientes fictícios
4. Implementação da geração de imagens simuladas
5. Criação do simulador de classificação automática
6. Desenvolvimento das views e URLs
7. Criação de comandos de gerenciamento
8. Testes dos endpoints
9. Documentação final do projeto

Esse planejamento ajudou a evitar confusão, retrabalho e erros durante o desenvolvimento.

4. DESCRIÇÃO DO PROCESSO DE DESENVOLVIMENTO

O desenvolvimento do projeto foi realizado de forma incremental, ou seja, **uma parte por vez**, testando cada funcionalidade antes de seguir para a próxima.

Inicialmente, foi criado o projeto Django e os aplicativos necessários, como o aplicativo do simulador.

Em seguida, foi implementado o **modelo de paciente**, definindo os campos essenciais como nome, CPF, sexo, email e histórico médico.

Após isso, foi utilizada a biblioteca **Faker** para gerar dados fictícios realistas, permitindo a criação automática de pacientes no banco de dados.

Na sequência, foram desenvolvidas funções para gerar **imagens de termografia simuladas**, utilizando NumPy e OpenCV. Essas imagens não representam exames reais, mas simulam padrões térmicos comuns encontrados em exames médicos.

Também foi implementado um **gerador de respostas aleatórias**, simulando um classificador inteligente inspirado no WEKA. Esse gerador retorna:

- Um possível diagnóstico
- Um nível de confiança
- Um tempo de processamento simulado

Posteriormente, foram criadas **views e URLs**, permitindo acessar essas funcionalidades diretamente pelo navegador ou por requisições HTTP.

Por fim, foi criado um **comando de gerenciamento personalizado**, possibilitando inicializar o banco de dados com dados fictícios de forma rápida e automatizada.

5. POSSÍVEIS DIFICULDADES ENCONTRADAS

Durante o desenvolvimento, algumas dificuldades puderam ser identificadas, como:

- Entendimento inicial da estrutura de pastas do Django
- Diferença entre services, views e urls
- Configuração correta das rotas (URLs)
- Erros comuns ao acessar endpoints inexistentes
- Compreensão de como comandos de gerenciamento funcionam
- Ajustes na geração de imagens simuladas para evitar erros

Essas dificuldades foram resolvidas com testes constantes, leitura da documentação e organização do código.

6. POSSÍVEIS MELHORIAS

Apesar do projeto atender aos objetivos propostos, diversas melhorias podem ser implementadas no futuro, como:

- Integração com um classificador real de Machine Learning
- Armazenamento das imagens no banco de dados ou em nuvem
- Criação de uma interface gráfica (frontend)
- Implementação de autenticação e permissões
- Criação de relatórios automáticos em PDF
- Inclusão de testes automatizados
- Uso de API REST com Django Rest Framework

Essas melhorias tornariam o sistema mais robusto e próximo de uma aplicação real.

7. CONCLUSÃO

O desenvolvimento deste simulador permitiu compreender, de forma prática, como funciona a criação de sistemas web voltados para a área da saúde utilizando Django.

Mesmo utilizando dados e imagens fictícias, o projeto simula situações reais encontradas em sistemas médicos, respeitando questões éticas e legais.

Além disso, o projeto contribuiu para o aprendizado sobre:

- Organização de projetos Django
- Geração de dados fictícios
- Processamento básico de imagens
- Simulação de sistemas inteligentes
- Criação de APIs e comandos personalizados